

Safora Kids AI: Adaptive Safety Learning Platform Report

Abstract

Safora Kids AI is a web-based adaptive learning platform designed to help children learn practical safety knowledge through daily quiz sessions, personalized question selection, spaced repetition, mastery tracking, feedback, and administrative analytics. The system focuses on five safety domains: traffic and road safety, natural disaster preparedness, household and occupational hazards, basic first aid, and good habits and hygiene. It uses a modern full-stack web architecture built with Next.js, React, TypeScript, Prisma, PostgreSQL, Tailwind CSS, Recharts, JWT-based authentication, and rule-based AI concepts for adaptive learning.

The core educational problem addressed by the project is that children learn safety concepts at different speeds, forget previously learned concepts over time, and need repeated practice in weak areas. A static quiz system cannot respond to individual mistakes or learning gaps. Safora Kids AI solves this by recording attempts, calculating mastery scores, scheduling reviews, selecting adaptive questions, and showing both student and administrator dashboards. The platform also includes a final assessment process that determines whether a student has achieved enough consistent mastery to be considered a “Safora Safety Champion.”

This report documents the problem statement, methodology, technical approach, AI concepts, implementation architecture, findings, conclusions, recommendations, and references. Mermaid diagram code is included for system architecture, data flow, adaptive question selection, spaced repetition, database relationships, and user workflow.

1. Problem Statement

Children need safety education that is practical, memorable, and age appropriate. Important topics such as road crossing, emergency response, fire drills, first aid, flood safety, hygiene, and safe household behavior are often taught through one-time lessons or general classroom instruction. These approaches can create awareness, but they may not reliably measure whether a child has retained the concept or can apply it in a scenario.

The main problem is the lack of a personalized and measurable safety-learning platform for children. Traditional quiz applications usually present the same questions to every learner, regardless of their past performance. They may show a score at the end of a quiz, but they often do not track long-term mastery, identify weak concepts, or schedule repeated review. A child who repeatedly misses questions about flood safety should receive more targeted practice in that concept, while a child who has already mastered road-safety behavior should be

challenged at an appropriate level and reviewed later to maintain retention.

Safora Kids AI addresses this problem by implementing an adaptive learning platform that:

1. Presents daily question sessions rather than a single static test.
2. Covers several safety modules relevant to children.
3. Tracks each answer attempt, score, and session result.
4. Calculates mastery by module.
5. Identifies repeated mistakes and weak concepts.
6. Uses spaced repetition to schedule future review.
7. Selects adaptive questions using a rule-based heuristic.
8. Provides immediate feedback and explanations after each answer.
9. Gives students visual progress indicators, earned badges, weak topics, and assessment results.
10. Gives administrators analytics about students, questions, weak concepts, AI selection behavior, retention, and pass status.

The system is therefore not only a quiz application. It is a learning loop that uses data from student behavior to decide what the student should practice next.

2. Project Scope

The scope of the Safora Kids AI platform includes student learning, admin monitoring, question management, adaptive selection, spaced repetition, and assessment. The project implementation includes:

- Student registration, login, and protected sessions.
- Admin login and protected admin pages.
- Daily learning sessions with 25 questions.
- Alternating daily question patterns to balance safety modules.
- Adaptive question selection based on mastery, mistakes, review timing, and difficulty.
- Attempt logging for every answer.
- Immediate answer feedback with explanation.
- Module mastery updates after every attempt.
- Question schedule updates for review intervals.
- User dashboard with mastery, score trend, concept depth, badges, and weak topics.
- Admin dashboard with student status, question distribution, weak concepts, AI accuracy, retention success, and due reviews.
- Question seed data with safety concepts, learning objectives, explanations, difficulty, and concept depth.
- Final assessment logic based on completed sessions, recent score average, high-score consistency, and weak module checks.

The project currently uses a rule-based adaptive engine. The data model also

includes an AI selection log and a strategy field that can distinguish between rule-based fallback and a possible Gemini/API-supported strategy in the future.

3. Methodology

The project follows an applied software engineering methodology combining requirements analysis, domain modeling, full-stack implementation, adaptive algorithm design, data persistence, analytics, and iterative user-interface development.

3.1 Requirement Analysis

The learning need was broken into student, teacher/admin, and system requirements.

Student requirements:

- A student should be able to register and log in.
- A student should receive one daily learning session when eligible.
- A student should answer multiple-choice safety questions.
- A student should receive immediate feedback and explanation.
- A student should see progress, mastery, badges, score history, and weak topics.
- A student should receive adaptive review instead of random repetition.

Administrator requirements:

- An admin should monitor all students.
- An admin should view pass, fail, and in-progress status.
- An admin should inspect weak concepts across students.
- An admin should view question distributions by module, difficulty, and concept depth.
- An admin should view adaptive selection behavior and retention indicators.
- An admin should configure off days for daily sessions.

System requirements:

- The system must store users, questions, sessions, attempts, mastery records, review schedules, AI selection logs, and application settings.
- The system must authenticate users securely.
- The system must protect student and admin routes.
- The system must calculate mastery and update spaced repetition schedules after attempts.
- The system must select adaptive questions using recorded performance.
- The system must generate dashboards from stored analytics data.

3.2 Domain Modeling

The problem domain was modeled around learning entities:

- **User**: a student or admin account.
- **Question**: a safety question with module, subtopic, difficulty, explanation, learning objective, concept, and depth.
- **Session**: a daily quiz session containing selected question IDs and session results.
- **Attempt**: a student's answer to a question.
- **ModuleMastery**: a calculated mastery score for each student and module.
- **QuestionSchedule**: spaced repetition data for when a question should be reviewed again.
- **AISelectionLog**: audit record of adaptive question-selection behavior.
- **AppSetting**: configurable application setting such as off days.

This structure lets the application support both operational behavior and analytics. For example, the **Attempt** table supports answer feedback, mastery updates, weak concept analysis, AI accuracy calculation, and score trends.

3.3 System Design Approach

The system was designed as a Next.js full-stack application. The App Router is used for pages and API routes. React components render the student and admin interfaces. Prisma provides typed database access to PostgreSQL. TypeScript is used across the application to reduce implementation errors and make data contracts clearer.

The design emphasizes a closed learning loop:

1. The student starts a daily session.
2. The system selects questions from normal module quotas, due reviews, and adaptive selection.
3. The student answers questions.
4. The system records attempts and updates mastery.
5. The system schedules the next review interval.
6. The dashboard shows progress and weak areas.
7. The next session uses the updated learning record to personalize question selection.

3.4 Adaptive Learning Methodology

The adaptive methodology is based on a weighted scoring heuristic. Each eligible question receives a priority score based on:

- Low module mastery: questions from weaker modules receive higher priority.
- Recent mistakes: concepts recently answered incorrectly are prioritized.

- Due review timing: questions not seen recently or due for review receive higher priority.
- Difficulty matching: question difficulty is matched to the learner’s current mastery level.
- Repeat priority: important or repeat-worthy questions can be weighted more strongly.

This approach is practical because it is explainable, deterministic, and easy to audit. It does not require a large training dataset. It applies AI concepts such as personalization, recommendation, learner modeling, mastery estimation, and adaptive sequencing without relying on a black-box model.

3.5 Spaced Repetition Methodology

The system uses a spaced repetition mechanism to support long-term retention. After every attempt, the platform updates a schedule for that user-question pair.

If the student answers correctly, the interval grows as repeated correctness increases:

- First correct answer: review after 3 active days.
- Second correct answer: review after 7 active days.
- Third correct answer: review after 14 active days.
- Four or more correct answers: review after 30 active days.

If the student answers incorrectly, the interval is reset to 1 active day. This means mistakes return quickly for review, while mastered items are reviewed less often. Off days are respected through the active-day calculation, so reviews are scheduled around days when sessions are unavailable.

3.6 Analytics Methodology

Analytics are generated from operational learning data rather than separate survey data. The system calculates:

- Average session score.
- Average mastery.
- Completed sessions.
- Pass, fail, and in-progress counts.
- Question distribution by module.
- Question distribution by difficulty.
- Question distribution by concept depth.
- Weak concepts from incorrect attempts.
- AI/adaptive question accuracy.
- Retention success from repeated scheduled questions.
- Due review counts.

This gives both individual and aggregate visibility into learning outcomes.

4. Technology Used

The project uses a modern TypeScript-based web stack.

4.1 Frontend

- **Next.js 15.3.3**: Used as the full-stack framework with App Router pages and API route handlers.
- **React 19.1.0**: Used for interactive user interfaces.
- **TypeScript 5.8.3**: Used for static typing.
- **Tailwind CSS 3.4.17**: Used for utility-first styling.
- **Lucide React**: Used for icons in the UI.
- **Recharts**: Used for dashboards and analytics charts.

4.2 Backend

- **Next.js API routes**: Used for authentication, sessions, answers, dashboards, analytics, question imports, and AI/adaptive operations.
- **Prisma 6.8.2**: Used as the ORM and migration tool.
- **PostgreSQL**: Used as the relational database provider.
- **Zod**: Used for request payload validation.
- **bcryptjs**: Used for password hashing.
- **jose**: Used for JWT signing and verification.

4.3 Data and Learning Logic

- **Prisma schema** defines users, questions, sessions, attempts, mastery, schedules, AI logs, and settings.
- **Seed scripts** create safety questions and demo student data.
- **Adaptive selection logic** ranks question candidates.
- **Mastery logic** updates module scores and review schedules.
- **Dashboard logic** aggregates student and admin analytics.

4.4 Development Tooling

- **pnpm**: Used as the package manager.
- **Prisma migrations**: Used for database schema evolution.
- **tsx**: Used to run TypeScript seed scripts.
- **ESLint/Next linting**: Included for code quality.

5. AI Concepts Used

Although the current implementation is rule-based, it uses several important AI and intelligent tutoring concepts.

5.1 Personalization

The system personalizes question selection for each student. Instead of giving every student the same fixed sequence, it considers the student's own mastery scores, answer history, recent mistakes, and review schedule.

5.2 Recommendation System

The adaptive selector behaves like a recommendation system. Its goal is to recommend the most useful next questions for a learner. Candidate questions are ranked using a weighted priority score, and the top-ranked questions are selected.

5.3 Learner Modeling

The platform builds a simple learner model through:

- Module mastery scores.
- Attempt history.
- Correct and incorrect counts.
- Weak concepts.
- Concept depth performance.
- Review schedule status.

This learner model helps the system estimate what the student knows and what should be practiced next.

5.4 Mastery Learning

Mastery learning is represented through `ModuleMastery`. The platform tracks whether the student is performing well enough in each safety module. Final completion is not based only on finishing sessions; it also checks recent performance and weak modules.

5.5 Spaced Repetition

Spaced repetition is used to strengthen memory retention. Correctly answered questions return after longer intervals, while incorrect questions return quickly.

5.6 Adaptive Sequencing

Adaptive sequencing determines the order and mix of learning items. Safora Kids AI combines due review questions, normal module quotas, and AI-selected questions into each daily session.

5.7 Explainability

The adaptive logic is explainable because it logs selected reasons, modules, concepts, question IDs, and strategy used. This is important in educational systems

because teachers and admins should understand why the system selected certain content.

5.8 Human-Centered AI

The system is designed for child safety education. The AI behavior is not used to replace teachers or parents; it supports them by identifying weak areas, recommending practice, and giving progress visibility.

6. System Architecture

The application follows a layered architecture:

- Presentation layer: Next.js pages and React components.
- API layer: Next.js route handlers.
- Business logic layer: session, mastery, adaptive selection, dashboard, settings, and auth libraries.
- Data access layer: Prisma client.
- Data storage layer: PostgreSQL database.

Mermaid: High-Level Architecture

flowchart TB

```
Student[Student User] --> UI[Next.js React UI]
Admin[Admin User] --> UI

UI --> Pages[App Router Pages]
Pages --> API[Next.js API Routes]

API --> Auth[Auth Service\nJWT + bcrypt]
API --> Session[Session Service]
API --> Adaptive[Adaptive Selection Service]
API --> Mastery[Mastery + Spaced Review Service]
API --> Dashboard[Dashboard + Analytics Service]
API --> Settings[App Settings Service]

Auth --> Prisma[Prisma ORM]
Session --> Prisma
Adaptive --> Prisma
Mastery --> Prisma
Dashboard --> Prisma
Settings --> Prisma

Prisma --> DB[(PostgreSQL Database)]
```


7. Data Model

The database schema supports a complete learning workflow. The key relationship is that a **User** has many **Session**, **Attempt**, and **ModuleMastery** records. A **Question** has many **Attempt** records. Each **Session** has many attempts and stores selected question IDs. **QuestionSchedule** stores personalized spaced repetition data per user and question.

Mermaid: Entity Relationship Diagram

```
erDiagram
    User ||--o{ Session : has
    User ||--o{ Attempt : makes
    User ||--o{ ModuleMastery : tracks
    Question ||--o{ Attempt : answered_in
    Session ||--o{ Attempt : contains
    User ||--o{ QuestionSchedule : schedules
    Question ||--o{ QuestionSchedule : reviewed_by
    User ||--o{ AiSelectionLog : generates

    User {
        string id
        string studentId
        string passwordHash
        string ageGroup
        string role
        int totalXp
        boolean isCompleted
        boolean passed
    }

    Question {
        string id
        string module
        string subtopic
        string difficulty
        string question
        json options
        int correctAnswer
        string explanation
        string masteryConcept
        int conceptDepth
        boolean aiEligible
    }

    Session {
```

```

    string id
    string userId
    datetime sessionDate
    int weekNumber
    string dayPattern
    int totalQuestions
    int correctCount
    float scorePercent
    boolean completed
    json questionIds
}

Attempt {
    string id
    string userId
    string sessionId
    string questionId
    int selectedAnswer
    boolean isCorrect
    json shownOptions
}

ModuleMastery {
    string id
    string userId
    string module
    float masteryScore
    int attemptsCount
    int correctCount
}

QuestionSchedule {
    string id
    string userId
    string questionId
    datetime nextDueDate
    int intervalDays
    float masteryLevel
    int timesSeen
    int timesCorrect
}

```

8. Application Workflow

The main student workflow starts with login and moves into the daily session. When a student starts a session, the backend checks whether today is an off day

and whether the student already completed today’s session. If not, the system creates or retrieves a session, selects questions, and returns the current question. After each answer, the system records the attempt, updates mastery, updates review scheduling, updates session progress, awards XP, and returns feedback.

Mermaid: Student Daily Session Flow

flowchart TD

```

A[Student Logs In] --> B[Open Dashboard]
B --> C{Can Start Today?}
C -- Off Day --> D[Show Rest Day Message]
C -- Session Completed --> E[Show Completed Status]
C -- Yes --> F[POST /api/session/start]
F --> G[Create or Load Daily Session]
G --> H[Select Due Reviews]
H --> I[Select Normal Module Questions]
I --> J[Select Adaptive Questions]
J --> K[Shuffle and Limit to 25 Questions]
K --> L[Show Current Question]
L --> M[Student Submits Answer]
M --> N[POST /api/session/answer]
N --> O[Validate and Record Attempt]
O --> P[Update Mastery]
P --> Q[Update Review Schedule]
Q --> R[Update Session Score and XP]
R --> S[Return Feedback + Explanation]
S --> T{More Questions?}
T -- Yes --> L
T -- No --> U[Show Result Page]

```

9. Adaptive Question Selection

The adaptive selection algorithm ranks all AI-eligible questions. For each candidate question, the algorithm calculates a priority score using four main factors:

- Mastery gap: lower mastery increases priority.
- Recent wrong concept: concepts recently missed receive strong priority.
- Days since last seen: questions not seen recently receive higher priority.
- Difficulty match: easier questions are prioritized when mastery is low; moderate matching is used otherwise.

The selected questions are logged in `AiSelectionLog`. This log records selected modules, concepts, question IDs, selected reason, and strategy used. The current implementation reports either `rule_based_fallback` or `gemini_or_rule_fallback` depending on whether a Gemini API key is configured, although the actual current selector is implemented as a deterministic ranking heuristic.

Mermaid: Adaptive Selection Logic

flowchart TD

```
A[Start Adaptive Selection] --> B[Load User Module Mastery]
B --> C[Load Question Review Schedules]
C --> D[Load Recent Wrong Attempts]
D --> E[Build Set of Weak Concepts]
E --> F[Load AI-Eligible Questions]
F --> G[For Each Candidate Question]
G --> H[Calculate Mastery Gap Score]
H --> I[Calculate Repeat Priority]
I --> J[Calculate Days Since Seen Score]
J --> K[Calculate Difficulty Match Score]
K --> L[Weighted Priority Score]
L --> M[Sort by Score Descending]
M --> N[Take Required Count]
N --> O[Extract Modules and Concepts]
O --> P[Create AI Selection Log]
P --> Q[Return Selected Questions]
```

Mermaid: Priority Score Formula

flowchart LR

```
A[Priority Score] --> B["45% Mastery Gap\n100 - module mastery"]
A --> C["25% Repeat Priority\nwrong concept or repeat weight"]
A --> D["15% Review Timing\ndays since last seen"]
A --> E["15% Difficulty Match\nbased on mastery level"]
```

10. Mastery and Spaced Repetition

The mastery update process runs after every answer. The platform updates the student's module-level mastery and the question-level review schedule.

Module mastery is calculated as:

$$\text{masteryScore} = \text{correctCount} / \text{attemptsCount} * 100$$

The schedule then determines when the same question should appear again. The interval is extended with repeated correct answers and reset after incorrect answers.

Mermaid: Mastery and Review Update Flow

flowchart TD

```
A[Answer Submitted] --> B{Is Answer Correct?}
B -- Yes --> C[Increment Correct Count]
B -- No --> D[Keep Correct Count Same]
C --> E[Increment Attempt Count]
```

```

D --> E
E --> F[Recalculate Module Mastery]
F --> G[Load Existing Question Schedule]
G --> H{Correct?}
H -- No --> I[Set Interval to 1 Active Day]
H -- Yes --> J{Times Correct}
J -- 1 --> K[Set Interval to 3 Active Days]
J -- 2 --> L[Set Interval to 7 Active Days]
J -- 3 --> M[Set Interval to 14 Active Days]
J -- 4+ --> N[Set Interval to 30 Active Days]
I --> O[Update Next Due Date]
K --> O
L --> O
M --> O
N --> O
O --> P[Persist QuestionSchedule]

```

11. Final Assessment Logic

The platform determines completion and pass status using recent learning performance. A student passes when:

- The final-week average score is at least 90%.
- At least four of the last five completed sessions are 90% or above.
- No safety module has mastery below 85%.

The system returns a status of “Safora Safety Champion” when passed, otherwise “Review Recommended.” This approach is stronger than a single final test because it checks consistency across several sessions and mastery across all modules.

Mermaid: Final Assessment Flow

flowchart TD

```

A[Run Final Assessment] --> B[Load Last 20 Completed Sessions]
B --> C[Load Module Masteries]
C --> D[Take Last 5 Sessions]
D --> E[Calculate Final Average]
E --> F[Count Last 5 Sessions Above 90]
F --> G[Find Modules Below 85 Mastery]
G --> H{Average >= 90?}
H -- No --> I[Review Recommended]
H -- Yes --> J{At Least 4 of 5 Above 90?}
J -- No --> I
J -- Yes --> K{Any Weak Modules?}
K -- Yes --> I
K -- No --> L[Safora Safety Champion]

```

```
I --> M[Update User Completion Status]
L --> M
```

12. Findings

12.1 The System Implements a Complete Learning Loop

The platform does more than display questions. It creates a cycle where learning activity produces data, data updates the learner model, and the learner model affects future content. This is the foundation of an adaptive learning system.

The loop is visible across the implementation:

- Sessions are created daily.
- Questions are selected from quotas, due reviews, and adaptive ranking.
- Attempts are stored.
- Mastery is recalculated.
- Review schedules are updated.
- Dashboards summarize progress.
- Final assessment uses historical performance.

12.2 The Adaptive Engine Is Explainable

The adaptive selector is intentionally transparent. It does not depend on an opaque model. The weighted scoring approach can be explained to administrators and adjusted as more evidence is collected. For a child-focused educational system, this is a practical and responsible design choice.

12.3 The Database Schema Supports Analytics

The schema captures both current state and historical behavior. `Attempt` records make it possible to analyze weak concepts, response accuracy, score trends, and AI-selected question performance. `QuestionSchedule` supports retention analysis. `AiSelectionLog` provides auditability for adaptive decisions.

12.4 The Learning Content Is Structured for Adaptivity

Questions include module, subtopic, difficulty, learning objective, mastery concept, repeat priority, AI eligibility, and concept depth. These fields allow the system to choose questions intelligently. Without this metadata, adaptive selection would be limited to random or score-only logic.

12.5 Spaced Repetition Is Well Aligned With Safety Learning

Safety knowledge must be retained, not only recognized once. The review schedule helps reinforce important concepts over time. Resetting the interval after incorrect answers ensures that weak concepts are revisited quickly.

12.6 The Admin Dashboard Turns Learning Data Into Action

The admin dashboard identifies weak concepts, question coverage, pass/fail status, average mastery, AI accuracy, retention success, and due reviews. This helps educators decide where classroom reinforcement may be needed.

12.7 The Current AI Is Rule-Based Rather Than Generative

The code includes a strategy marker that can refer to a Gemini-enabled path, but the implemented selector is a deterministic rule-based heuristic. This is not a weakness by itself; it is appropriate for early-stage adaptive learning because it is predictable, testable, and safe. However, future work could add a generative or model-assisted layer for explanations, question generation, or deeper personalization.

12.8 The System Has a Clear Path for Improvement

The architecture already separates session logic, mastery logic, AI selection, dashboard aggregation, auth, and settings. This separation makes future improvements feasible without rewriting the entire application.

13. Limitations

The current implementation has several limitations that should be considered.

13.1 Rule Weights Are Static

The adaptive priority weights are hard-coded. The system does not yet learn whether the weights are optimal for different age groups, topics, or student profiles.

13.2 Mastery Calculation Is Simple

Current mastery is based on correct answers divided by attempts. This is understandable but does not account for recency, question difficulty, concept depth, guessing probability, or response time.

13.3 No Full Generative AI Integration Yet

The platform logs a possible Gemini strategy when an API key exists, but the actual selection logic remains rule-based. There is not yet an LLM-based explanation generator, misconception detector, or content generator.

13.4 Age Adaptation Can Be Expanded

Questions contain age groups, and users have age groups, but deeper age-specific adaptation can be improved. Younger learners may need simpler wording, more images, audio, or fewer questions.

13.5 Learning Outcome Validation Requires Real User Data

Demo data exists, but real effectiveness should be measured with actual students over time. The system can collect the needed data, but learning-impact evaluation requires deployment and observation.

14. Conclusion

Safora Kids AI is a strong example of a practical adaptive learning platform for child safety education. It uses a modern web stack and a thoughtful data model to create personalized daily learning sessions, track mastery, schedule review, provide feedback, and support administrator oversight.

The project successfully connects educational goals with implementation logic. Safety topics are organized into modules and concepts. Questions include learning objectives, explanations, difficulty, and depth. Student attempts are captured in detail. Mastery and spaced repetition are updated automatically. Adaptive question selection uses low mastery, repeated mistakes, due review timing, and difficulty matching to recommend useful questions. Dashboards transform the resulting data into meaningful progress and analytics.

The current AI approach is rule-based and explainable. This is suitable for a safety-focused learning system because decisions can be audited and adjusted. The platform provides a foundation for future AI enhancements, including generative question creation, personalized explanations, misconception classification, and more advanced mastery estimation.

Overall, the project demonstrates how AI concepts can be applied responsibly without requiring a complex black-box model. The most important achievement is the creation of a feedback-driven learning cycle where every answer improves the next learning experience.

15. Recommendations

15.1 Improve the Mastery Model

The mastery calculation should eventually include recency, difficulty, concept depth, and confidence. A weighted mastery model would better represent real learning than a simple correct/attempt ratio.

15.2 Add More Age-Specific Adaptation

The system should use age group more directly in question selection, interface design, explanation style, and session length. Younger students may benefit from shorter sessions and more visual support.

15.3 Add LLM-Assisted Explanation Support

A generative AI model could produce personalized explanations after wrong answers. The explanation should remain constrained by approved safety content and reviewed templates to avoid unsafe or confusing responses.

15.4 Add Misconception Detection

Wrong answers can reveal specific misconceptions. For example, choosing to run during a fire drill indicates a different misconception than hiding in a classroom. The system could map distractors to misconception categories.

15.5 Add Teacher Intervention Recommendations

The admin dashboard could recommend interventions such as “review flood safety with Grade 4” or “assign first-aid practice to students below 70% mastery.”

15.6 Validate With Real Learners

The platform should be tested with students over multiple weeks. Important metrics include retention improvement, session completion rate, weak concept reduction, and teacher satisfaction.

15.7 Strengthen Security for Production

For production deployment, the JWT secret must be securely configured, environment variables must be protected, and admin access should be carefully controlled. Rate limiting and audit logs should be added for sensitive endpoints.

15.8 Add Content Review Workflow

Because the platform teaches safety concepts to children, new or imported questions should pass through a review workflow before becoming active.

15.9 Expand Accessibility

The platform should support keyboard navigation, screen reader labels, high-contrast modes, and possibly audio reading for younger learners.

15.10 Convert This Markdown Report to DOCX

The requested output file is `report.md`. To produce a DOCX version from this markdown document, a tool such as Pandoc can be used:

```
pandoc report.md -o Safora_Kids_AI_Report.docx
```

16. Additional Mermaid Diagrams

16.1 Admin Analytics Flow

```
flowchart TD
    A[Admin Opens Dashboard] --> B[Require Admin Auth]
    B --> C[Load Students]
    B --> D[Load Questions]
    B --> E[Load Sessions]
    B --> F[Load Attempts]
    B --> G[Load AI Logs]
    B --> H[Load Review Schedules]
    C --> I[Calculate Student Status]
    D --> J[Question Distribution]
    E --> K[Average Score + Score Bands]
    F --> L[Weak Concepts]
    G --> M[AI Accuracy + Selection Reasons]
    H --> N[Retention Success + Due Reviews]
    I --> O[Admin Dashboard Charts]
    J --> O
    K --> O
    L --> O
    M --> O
    N --> O
```

16.2 Authentication Flow

```
sequenceDiagram
    participant User
    participant UI as Login UI
    participant API as Auth API
    participant DB as PostgreSQL
    participant Cookie as HTTP-only Cookie

    User->>UI: Enter studentId and password
    UI->>API: POST login request
    API->>DB: Find user by studentId
    DB-->>API: User record with password hash
    API->>API: Verify password with bcrypt
    API->>API: Sign JWT with jose
    API->>Cookie: Set safora_token
    API-->>UI: Login success
    UI-->>User: Redirect to dashboard/admin
```

16.3 Session Answer Sequence

```
sequenceDiagram
    participant Student
    participant UI as SessionPlayer
    participant API as /api/session/answer
    participant DB as Prisma/PostgreSQL
    participant Mastery as Mastery Service

    Student->>UI: Select answer
    UI->>API: Submit sessionId, questionId, selectedAnswer
    API->>DB: Load session and question
    API->>DB: Create Attempt
    API->>Mastery: updateMasteryAfterAttempt
    Mastery->>DB: Upsert ModuleMastery
    Mastery->>DB: Upsert QuestionSchedule
    API->>DB: Update Session counts and score
    API->>DB: Increment XP when applicable
    API-->>UI: Feedback, correct answer, explanation
    UI-->>Student: Show result and explanation
```

16.4 Deployment View

```
flowchart LR
    Browser[Browser] --> Next[Next.js Application Server]
    Next --> API[Route Handlers]
    API --> Prisma[Prisma Client]
    Prisma --> Postgres[(PostgreSQL)]
    Next --> Static[Static Assets + CSS]
    API --> Env[Environment Variables\nDATABASE_URL, JWT_SECRET, GEMINI_API_KEY]
```

References

1. Project source code: `package.json`, `prisma/schema.prisma`, `src/lib/ai.ts`, `src/lib/session.ts`, `src/lib/mastery.ts`, `src/lib/dashboard.ts`, `src/lib/auth.ts`, and related API routes.
2. Next.js Documentation. <https://nextjs.org/docs>
3. React Documentation. <https://react.dev>
4. TypeScript Documentation. <https://www.typescriptlang.org/docs>
5. Prisma Documentation. <https://www.prisma.io/docs>
6. PostgreSQL Documentation. <https://www.postgresql.org/docs>
7. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
8. Recharts Documentation. <https://recharts.org>
9. Zod Documentation. <https://zod.dev>
10. jose Documentation. <https://github.com/panva/jose>
11. bcrypt password hashing package documentation. <https://github.com/dcodeIO/bcrypt.js>

12. Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., and Rohrer, D. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 2006.
13. Bloom, B. S. Learning for Mastery. *Evaluation Comment*, 1968.
14. Corbett, A. T., and Anderson, J. R. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 1995.
15. Brusilovsky, P. Adaptive and Intelligent Web-based Educational Systems. *International Journal of Artificial Intelligence in Education*, 2003.